Internship Project

Name: Aditya Singh

Domain: Machine Learning

Project Title: Credit Card Fraud Detection Using Logistic Regression

Submission date: 11/10/2025

Abstract:

This project aims to detect fraudulent credit card transactions using machine learning techniques. By analyzing transaction patterns and applying logistic regression, the model identifies anomalies that may indicate fraud. The dataset used is highly imbalanced, making it a real-world challenge for classification algorithms. The project demonstrates how data preprocessing, model training, and evaluation can be combined to build a reliable fraud detection system.

Objective:

- To build a binary classification model that distinguishes between fraudulent and legitimate credit card transactions.
- To evaluate the performance of logistic regression on an imbalanced dataset.
- To understand the impact of preprocessing and feature selection on model accuracy.

Methodology:

1. Data Collection
   - Dataset sourced from Kaggle: Credit Card Fraud Detection
2. Data Preprocessing
   - Load CSV data using Pandas
   - Inspect and clean data
   - Normalize features if needed
   - Handle class imbalance (optional: SMOTE, undersampling)
3. Train-Test Split
   - Split data into training and testing sets (80/20)
4. Model Building
   - Use Logistic Regression from sklearn.linear_model
5. Evaluation

- Accuracy Score
- Confusion Matrix
- Precision, Recall, F1 Score

Tools Used:

- **Python –>** Programming Language
- **Pandas ->** Data manipulation
- **Numpy ->** Numerical operations
- **Scikit-learn ->** ML models and evaluation metrics
- **Google Colab ->** Interactive coding environment

Step by step execution:

1. Import Libraries

```
[349]
✓ 0s
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

2. Load Dataset

```
# loading Dataset to Pandas dataframe
credit_card_data = pd.read_csv('/content/creditcard.csv')
```

3. Printing first 5 rows

```
# first 5 rows of the dataset
credit_card_data.head()
```

	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V21	V22	V23	V24	V25	V26	
0	0.0	-1.359807	-0.072781	2.536347	1.378155	-0.338321	0.462388	0.239599	0.098698	0.363787	...	-0.018307	0.277838	-0.110474	0.066928	0.128539	-0.189115	0.
1	0.0	1.191857	0.266151	0.166480	0.448154	0.060018	-0.082361	-0.078803	0.085102	-0.255425	...	-0.225775	-0.638672	0.101288	-0.339846	0.167170	0.125895	-0.
2	1.0	-1.358354	-1.340163	1.773209	0.379780	-0.503198	1.800499	0.791461	0.247676	-1.514654	...	0.247998	0.771679	0.909412	-0.689281	-0.327642	-0.139097	-0.
3	1.0	-0.966272	-0.185226	1.792993	-0.863291	-0.010309	1.247203	0.237609	0.377436	-1.387024	...	-0.108300	0.005274	-0.190321	-1.175575	0.647376	-0.221929	0.
4	2.0	-1.158233	0.877737	1.548718	0.403034	-0.407193	0.095921	0.592941	-0.270533	0.817739	...	-0.009431	0.798278	-0.137458	0.141267	-0.206010	0.502292	0.

5 rows × 31 columns

4. Getting the information of the dataset

```
# information of the dataset
credit_card_data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
 #   Column  Non-Null Count   Dtype  
---  ------  --------------   -----  
 0   Time    284807 non-null  float64
 1   V1      284807 non-null  float64
 2   V2      284807 non-null  float64
 3   V3      284807 non-null  float64
 4   V4      284807 non-null  float64
 5   V5      284807 non-null  float64
 6   V6      284807 non-null  float64
 7   V7      284807 non-null  float64
 8   V8      284807 non-null  float64
 9   V9      284807 non-null  float64
 10  V10     284807 non-null  float64
 11  V11     284807 non-null  float64
 12  V12     284807 non-null  float64
 13  V13     284807 non-null  float64
 14  V14     284807 non-null  float64
 15  V15     284807 non-null  float64
 16  V16     284807 non-null  float64
 17  V17     284807 non-null  float64
 18  V18     284807 non-null  float64
 19  V19     284807 non-null  float64
 20  V20     284807 non-null  float64
 21  V21     284807 non-null  float64
 22  V22     284807 non-null  float64
 23  V23     284807 non-null  float64
 24  V24     284807 non-null  float64
 25  V25     284807 non-null  float64
 26  V26     284807 non-null  float64
 27  V27     284807 non-null  float64
 28  V28     284807 non-null  float64
 29  Amount  284807 non-null  float64
 30  Class   284807 non-null  int64  
dtypes: float64(30), int64(1)
memory usage: 67.4 MB
```

6. Checking for the missing values

```
# checking the number of the missing values in each column
credit_card_data.isnull().sum()
```

	0
Time	0
V1	0
V2	0
V3	0
V4	0
V5	0
V6	0
V7	0
V8	0
V9	0
V10	0
V11	0
V12	0
V13	0
V14	0
V15	0
V16	0
V17	0
V18	0
V19	0
V20	0
V21	0
V22	0
V23	0
V24	0
V25	0
V26	0
V27	0
V28	0
Amount	0
Class	0

dtype: int64

7. Checking the distribution of the legit and fraud transaction

```
# checking the distribution of legit and fraud transactions in the dataset
credit_card_data['Class'].value_counts()
```

Class	count
0	284315
1	492

dtype: int64

8. Seperating the data for analysis

```
# separating the data for the analysis
legit = credit_card_data[credit_card_data.Class == 0]
fraud = credit_card_data[credit_card_data.Class == 1]

print(legit.shape)
print(fraud.shape)
```

```
(284315, 31)
(492, 31)
```

9. Finding the statistical measures of the data

```
# statistical measures of the data
legit.Amount.describe()
```

	Amount
count	284315.000000
mean	88.291022
std	250.105092
min	0.000000
25%	5.650000
50%	22.000000
75%	77.050000
max	25691.160000

dtype: float64

```
fraud.Amount.describe()
```

	Amount
count	492.000000
mean	122.211321
std	256.683288
min	0.000000
25%	1.000000
50%	9.250000
75%	105.890000
max	2125.870000

dtype: float64

10. Comparing both on the basis of mean

```
# compare the values for both of the transactions
credit_card_data.groupby('Class').mean()
```

Class	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V20	V21	V22	V23	V24
0	94838.202258	0.008258	-0.006271	0.012171	-0.007860	0.005453	0.002419	0.009637	-0.000987	0.004467	...	-0.000644	-0.001235	-0.000024	0.000070	0.000182
1	80746.806911	-4.771948	3.623778	-7.033281	4.542029	-3.151225	-1.397737	-5.568731	0.570636	-2.581123	...	0.372319	0.713588	0.014049	-0.040308	-0.105130

2 rows × 30 columns

11. Undersampling

Under-sampling

Build a sample dataset containig similar distribution of legit and fraud transaction

Number of fraud transactions -- 492

```
legit_sample = legit.sample(n=492)
```

12. Concatenating both datasets and comparing on the basis of mean

Concatenating two dataframes

```
new_dataset = pd.concat([legit_sample, fraud],axis=0)
```

```
new_dataset['Class'].value_counts()
```

Class	count
0	492
1	492

dtype: int64

```
new_dataset.groupby('Class').mean()
```

Class	Time	V1	V2	V3	V4	V5	V6	V7	V8	V9	...	V20	V21	V22	V23	V24
0	96644.723577	0.115773	-0.119701	-0.000311	-0.021577	-0.032846	-0.044786	0.029332	-0.014261	0.009563	...	0.025331	0.004425	0.019234	-0.026167	0.054591
1	80746.806911	-4.771948	3.623778	-7.033281	4.542029	-3.151225	-1.397737	-5.568731	0.570636	-2.581123	...	0.372319	0.713588	0.014049	-0.040308	-0.105130

2 rows × 30 columns

13. Splitting dataset into features and target

Splitting the data into features and targets

```
X = new_dataset.drop(columns='Class', axis=1)
Y = new_dataset['Class']
```

```
print(X)
```

```
            Time        V1        V2        V3        V4        V5        V6  \
202699  134450.0 -0.719922 -0.457426  1.055190  0.144376  0.102199  1.371500
181174  124858.0  0.024058  0.731933  0.128423 -0.794243  0.608320 -0.514292
24179    33104.0 -1.275329  1.600861  1.488741  2.900724 -0.432104  0.233517
120879   75992.0 -0.913505  0.116143  1.636062 -1.769871 -0.899943 -0.471192
111286   72172.0 -1.568901  1.193556  1.991175 -2.051756  0.977886 -0.214633
...          ...       ...       ...       ...       ...       ...       ...
279863  169142.0 -1.927883  1.125653 -4.518331  1.749293 -1.566487 -2.010494
280143  169347.0  1.378559  1.289381 -5.004247  1.411850  0.442581 -1.326536
280149  169351.0 -0.676143  1.126366 -2.213700  0.468308 -1.120541 -0.003346
281144  169966.0 -3.113832  0.585864 -5.399730  1.817092 -0.840618 -2.943548
281674  170348.0  1.991976  0.158476 -2.583441  0.408670  1.151147 -0.096695

               V7        V8        V9  ...       V20       V21       V22  \
202699  0.403534  0.396322  0.131708  ...  0.447267  0.536606  1.467711
181174  0.812204  0.047484 -0.128508  ... -0.089152 -0.241887 -0.589065
24179   0.602361 -0.046705 -0.333362  ...  0.286870 -0.283375 -0.347572
120879 -0.511050  0.454210 -1.184656  ...  0.115552  0.507468  1.124759
111286  2.048346 -1.721638  2.194256  ...  0.931997 -0.383752  0.142537
...          ...       ...       ...  ...       ...       ...       ...
279863 -0.882850  0.697211 -2.064945  ...  1.252967  0.778584 -0.319189
280143 -1.413170  0.248525 -1.127396  ...  0.226138  0.370612  0.028234
280149 -2.234739  1.210158 -0.652250  ...  0.247968  0.751826  0.834108
```

```
              V23       V24       V25       V26       V27       V28  Amount
202699  0.434305 -0.966201 -0.839885  0.800862  0.166337  0.204023  201.80
181174  0.020948 -0.499637 -0.515423  0.154063  0.240399  0.080342    0.89
24179   0.115988  0.363422 -0.458909 -0.025059  0.276092  0.421763   65.22
120879 -0.251553 -0.101202  0.187470 -0.169219 -0.027006  0.016758   39.00
111286 -0.290053  0.016431 -0.519659  0.477640 -1.605668 -1.380345    1.46
...          ...       ...       ...       ...       ...       ...     ...
279863  0.639419 -0.294885  0.537503  0.788395  0.292680  0.147968  390.00
280143 -0.145640 -0.081049  0.521875  0.739467  0.389152  0.186637    0.76
280149  0.190944  0.032070 -0.739695  0.471111  0.385107  0.194361   77.89
281144 -0.456108 -0.183659 -0.328168  0.606116  0.884876 -0.253700  245.00
281674 -0.072173 -0.450261  0.313267 -0.289617  0.002988 -0.015309   42.53

[984 rows x 30 columns]
```

```
print(Y)
```

```
202699    0
181174    0
24179     0
120879    0
111286    0
         ..
279863    1
280143    1
280149    1
281144    1
281674    1
Name: Class, Length: 984, dtype: int64
```

14. Splitting into training and testing dataset

Split the data into Training data & Testing data

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, stratify=Y, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(984, 30) (787, 30) (197, 30)
```

15. Model training

Model Training

+ Code + Text

Logistic Regression

```
model = LogisticRegression()
```

```
# training the logistic regression model with Training data
model.fit(X_train, Y_train)
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression
  n_iter_i = _check_optimize_result(
```

LogisticRegression

LogisticRegression()

16. Model evaluation

Model Evaluation

```
# accuracy on training data
X_train_prediction = model.predict(X_train)
training_data_accuracy = accuracy_score(X_train_prediction, Y_train)
```

```
print('Accuracy on Training data : ', training_data_accuracy)
```

```
Accuracy on Training data :  0.9415501905972046
```

```
# accuracy on testing data
X_test_prediction = model.predict(X_test)
testing_data_accuracy = accuracy_score(X_test_prediction, Y_test)
```

```
print('Accuracy on Testing data : ', testing_data_accuracy)
```

```
Accuracy on Testing data :  0.9441624365482234
```

Final Output / Result:

- Accuracy: 0.944 (depending on preprocessing)
- Confusion Matrix: Shows true positives, false positives, etc.
- Model Insight: Logistic regression performs well but may struggle with class imbalance.

Conclusion:

This project successfully demonstrates how logistic regression can be applied to detect fraudulent transactions. Despite the dataset's imbalance, the model achieves high accuracy. Future improvements could include using ensemble methods like XGBoost or applying resampling techniques to improve recall on minority classes. The project highlights the importance of preprocessing and evaluation in building robust ML systems.